

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: MLA0304

COURSE NAME: Reinforcement learning for Environment and Agent

COURSE OUTCOME COVERED

CO3: Implement policy-based reinforcement learning methods to develop efficient solutions for complex environments. (BL2, BL3, BL6)

Assessment Tool Weightage: 30%

Dr DUMPALA PRASANTH

QUESTIONS: 10

Total Marks: 50

Annexure A

CODING CHALLENGE

Duration: 2hrs

1. Predict the Reinforcement Learning (RL) framework by describing the agent–environment interaction, emphasizing the importance of states, actions, and rewards, and discuss how cumulative reward influences decision-making. **(5 marks)**

2. Estimate how decision-making problems can be modeled as a Markov Decision Process (MDP), detailing the components of action space, transition probability, reward function, and discount factor. **(5 marks)**

3. Discuss the role of policy and value functions in RL, and examine how state-value and action-value functions contribute to evaluating long-term benefits of actions using the Bellman Equation. **(5 marks)**

4. Justify the exploration–exploitation trade-off in RL and compare strategies such as ϵ -greedy and softmax approaches in balancing learning efficiency. **(5 marks)**

5. Explain how reward shaping can accelerate learning in RL and illustrate its impact on preventing suboptimal policies. **(5 marks)**

6. Identify the challenges of applying RL in real-world robotics and analyze how safety, sample efficiency, and environment variability affect agent performance. **(5 marks)**

7. Analyze the influence of the discount factor (γ) on short-term versus long-term decision-making, and evaluate its role in episodic tasks. **(5 marks)**

8. Evaluate the effectiveness of model-free RL compared to model-based RL in dynamic environments, and differentiate their strengths and limitations. **(5 marks)**

9. Illustrate with an example how Q-learning updates the action-value function during training, and demonstrate its convergence behavior. **(5 marks)**

10. Differentiate between on-policy and off-policy learning methods in RL, and assess their advantages and limitations in practical applications. **(5 marks)**

Code of Conduct:

I (k varshan / 192325089) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

K Varshan

RUBRICS FOR CALCULATION

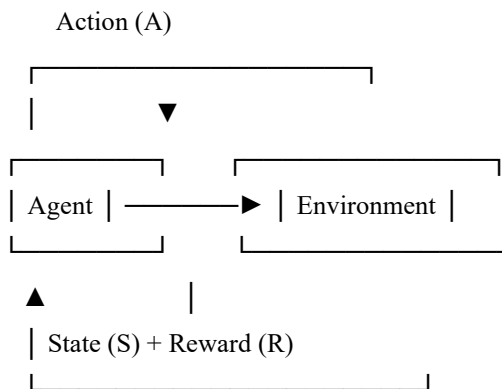
Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively
Confidence	15	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately

1. Reinforcement Learning Framework

Reinforcement Learning (RL) is a learning method in which an **agent interacts with an environment** and learns to make decisions by receiving rewards or penalties.

Agent–Environment Interaction

The interaction can be represented as:



The main components are:

State (S): Represents the current situation of the environment.

Action (A): A decision taken by the agent.

Reward (R): Feedback received after taking an action.

Environment: The system with which the agent interacts.

Policy (π): Strategy used by the agent to select actions.

Importance of State, Action and Reward

For example, in a robot navigation system:

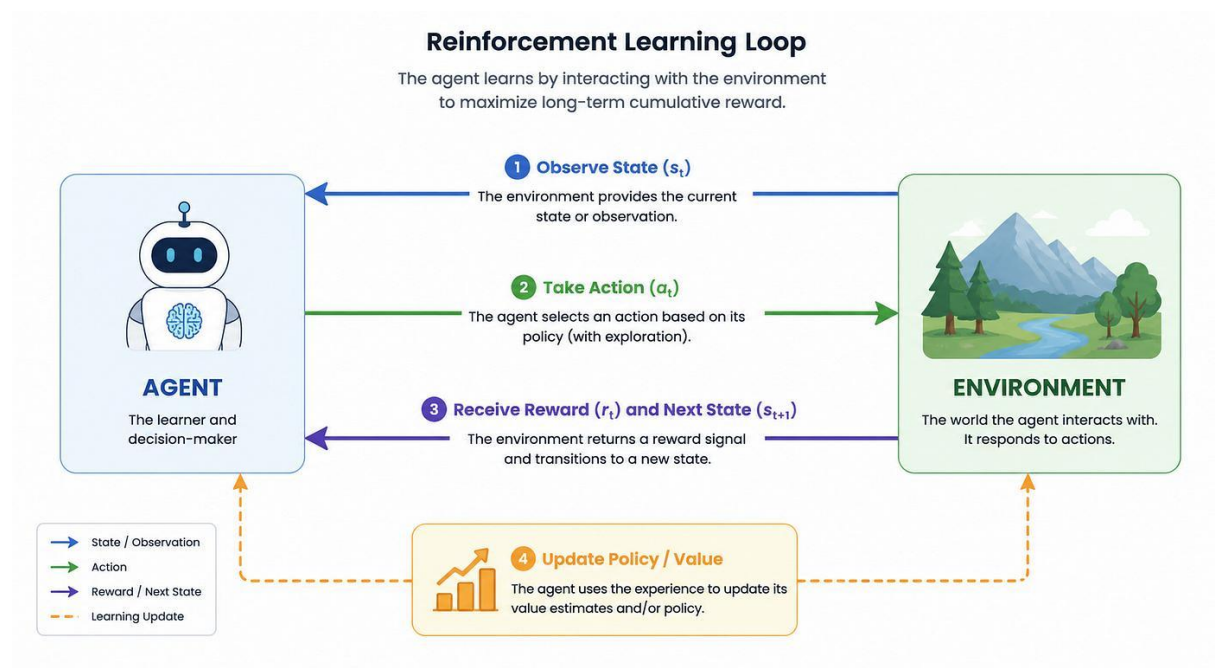
State → robot's current position.

Action → move left, right, forward, or backward.

Reward → +10 for reaching the destination and −10 for hitting an obstacle.

The agent tries to maximize the cumulative reward, rather than only the immediate reward.

Diagram



The return is:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

where γ is the discount factor.

Thus, cumulative reward encourages the agent to choose actions that provide better long-term benefits.

2. Decision-Making as a Markov Decision Process

A Markov Decision Process (MDP) is a mathematical framework used to model sequential decision-making problems.

An MDP is represented as:

$$[\text{MDP} = (S, A, P, R, \gamma)]$$

Components

1. State Space (S)

Set of all possible states.

Example: In a self-driving car, states may include road position, speed, traffic condition, etc.

2. Action Space (A)

Set of actions available to the agent.

Example:

$$A = \{\text{Accelerate, Brake, Turn Left, Turn Right}\}$$

3. Transition Probability P

It represents the probability of moving from one state to another after taking an action.

$$[P(s'|s,a)]$$

For example, if a robot moves forward, there may be an 80% probability of reaching the expected position and a 20% probability of moving to another position.

4. Reward Function R

It defines the immediate reward received after an action.

[
 $R(s,a,s')$
]

Example:

Reach destination $\rightarrow +100$

Move normally $\rightarrow +1$

Hit obstacle $\rightarrow -100$

5. Discount Factor γ

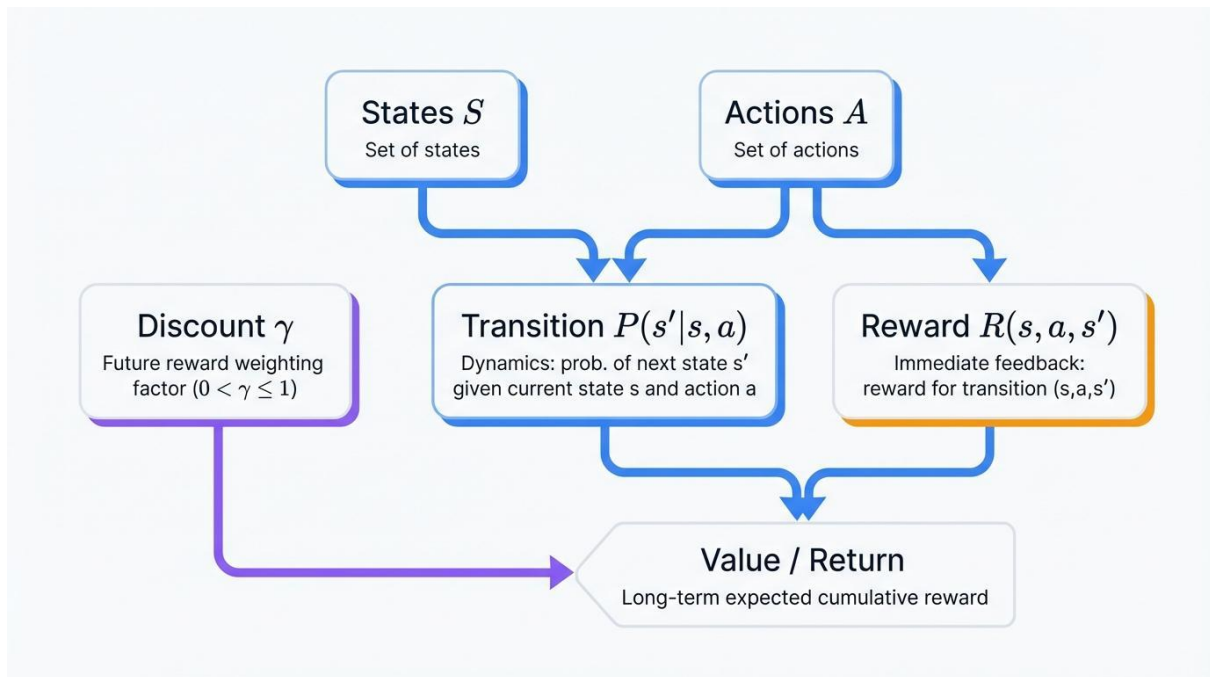
It determines the importance of future rewards.

[
 $0 \leq \gamma \leq 1$
]

A low γ emphasizes immediate rewards, while a high γ emphasizes future rewards.

Therefore, MDP provides a structured way to represent uncertain sequential decision-making.

Diagram



3. Policy and Value Functions

A **policy** defines how an RL agent selects an action based on the current state.

$$\pi(a|s) = P(A_t = a | S_t = s)$$

A policy can be deterministic or stochastic.

Value Functions

Value functions estimate the **long-term expected reward**.

1. State-Value Function

The state-value function measures how good it is for the agent to be in a particular state.

$$V^\pi(s) = E_\pi[G_t | S_t = s]$$

It answers:

"How valuable is this state?"

2. Action-Value Function

The action-value function measures the expected long-term reward for taking action **a** in state **s**.

$$Q^{\pi}(s,a) = E_{\pi}[G_t | S_t=s, A_t=a]$$

It answers:

"How good is this action in this state?"

Bellman Equation

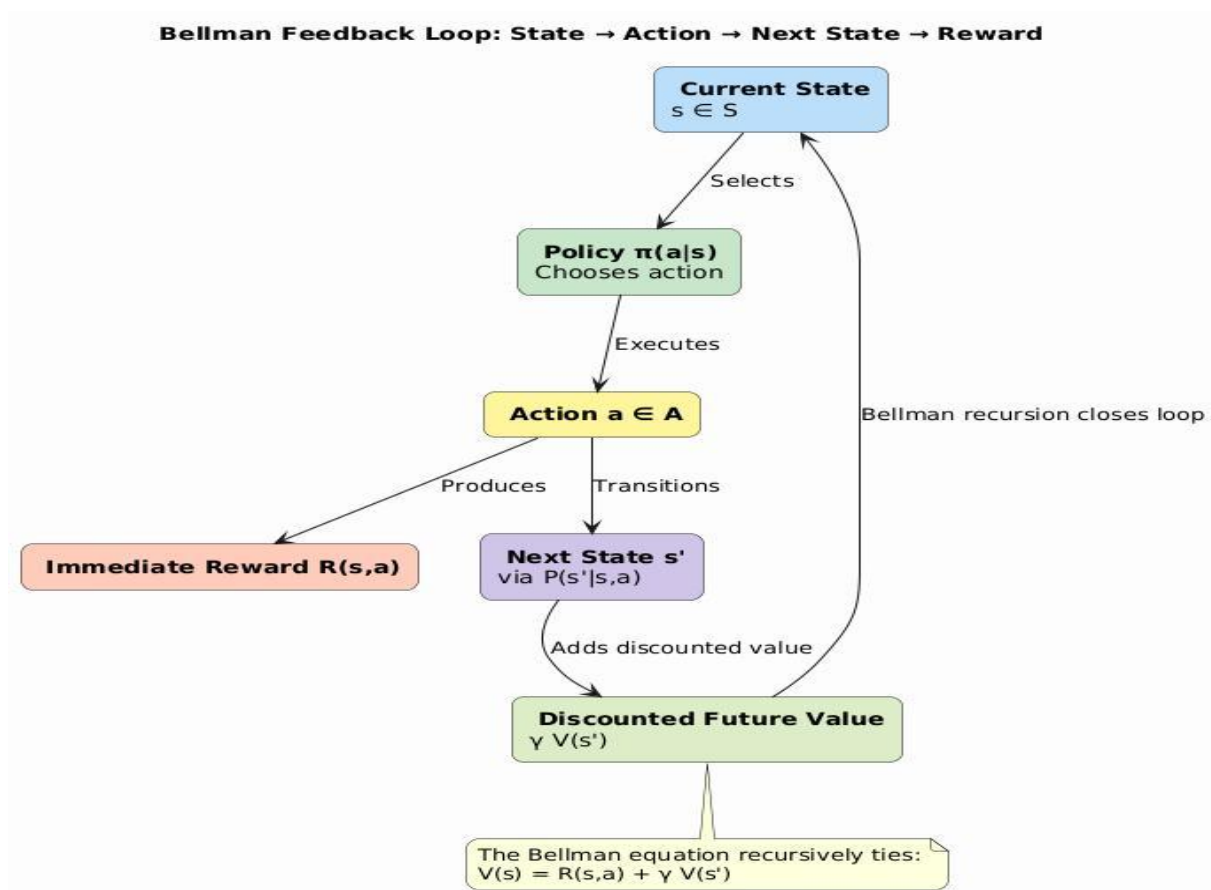
The Bellman equation expresses the value of a state in terms of its immediate reward and future value:

$$V^{\pi}(s) = \sum_a \pi(a|s) \left[R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^{\pi}(s') \right]$$

For Q-values:

$$Q^{\pi}(s,a) = R(s,a) + \gamma \sum_{s'} P(s'|s,a) V^{\pi}(s')$$

Diagram



4. Exploration–Exploitation Trade-off

The exploration–exploitation trade-off is one of the major challenges in RL.

Exploration: Trying new actions to discover potentially better rewards.

Exploitation: Choosing the action currently believed to provide the highest reward.

ϵ -Greedy Strategy

In ϵ -greedy:

With probability ϵ , choose a random action.

With probability $1-\epsilon$, choose the best-known action.

Example:

[
 $\epsilon=0.1$
]

means:

10% → Explore

90% → Exploit

Softmax Strategy

Softmax selects actions based on their estimated Q-values.

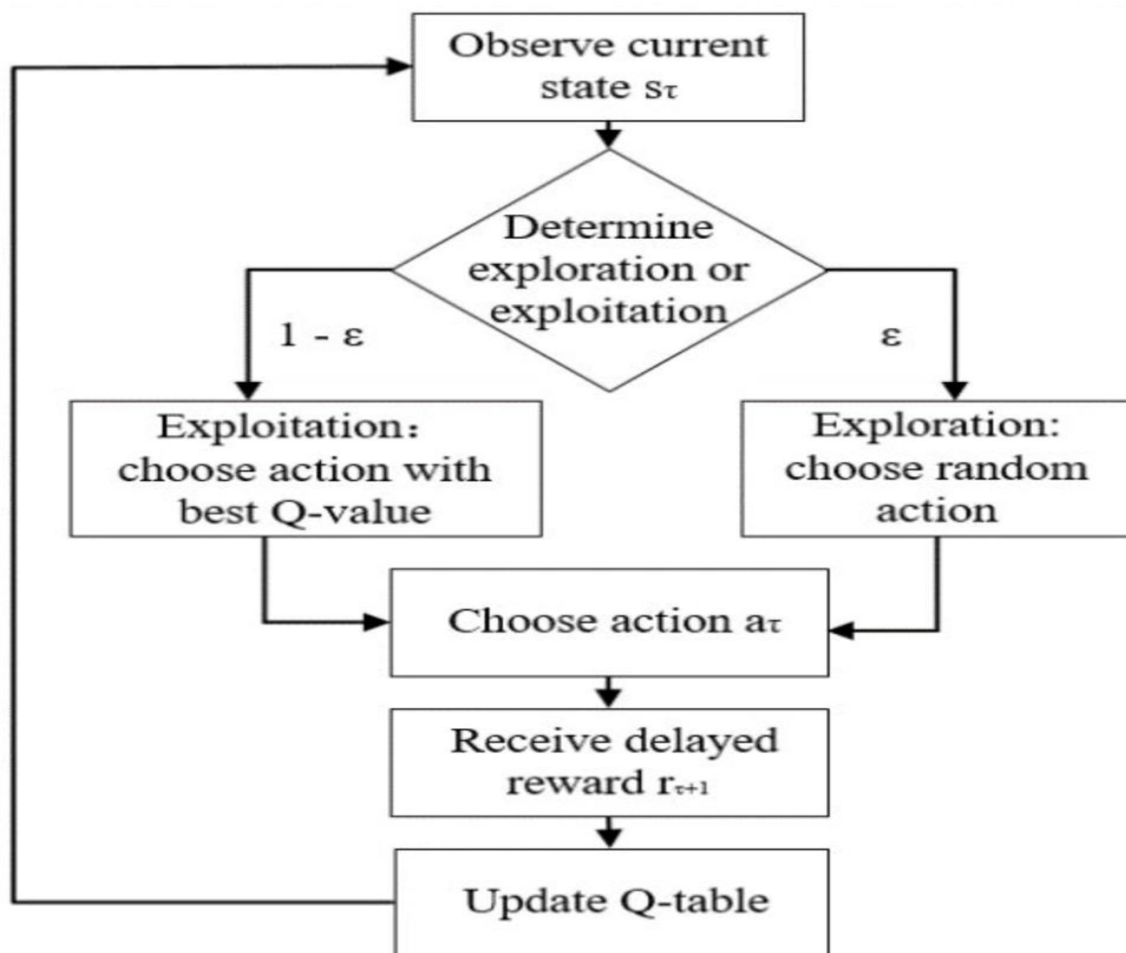
[
 $P(a|s)=$
 $\frac{e^{Q(s,a)/\tau}}{\sum_b e^{Q(s,b)/\tau}}$
]

where τ is the temperature parameter.

High τ → more exploration.

Low τ → more exploitation.

Diagram



Comparison

ϵ -Greedy

Simple to implement

Random exploration

May select a very poor action randomly

Computationally simple

Softmax

More gradual action selection

Probability based on Q-values

Gives higher probability to better actions

Slightly more computationally expensive

Both methods help the agent learn efficiently by balancing exploration and exploitation.

5. Reward Shaping in Reinforcement Learning

Reward shaping is the process of providing additional rewards or penalties to guide an RL agent toward desirable behavior.

Normally, an agent may receive a reward only after completing a task. This can make learning slow.

Example: Robot Navigation

Suppose a robot must reach a destination.

Without reward shaping:

Reach destination $\rightarrow +100$

Other actions $\rightarrow 0$

The robot receives useful feedback only at the end.

With reward shaping:

Move closer to goal $\rightarrow +5$

Move away from goal $\rightarrow -2$

Reach goal $\rightarrow +100$

Hit obstacle $\rightarrow -100$

The agent receives more frequent feedback.

Benefits

Accelerates learning.

Provides guidance during training.

Reduces unnecessary exploration.

Helps the agent discover useful strategies faster.

Preventing Suboptimal Policies

Poorly designed rewards can cause unintended behavior.

For example:

Goal reward = +100

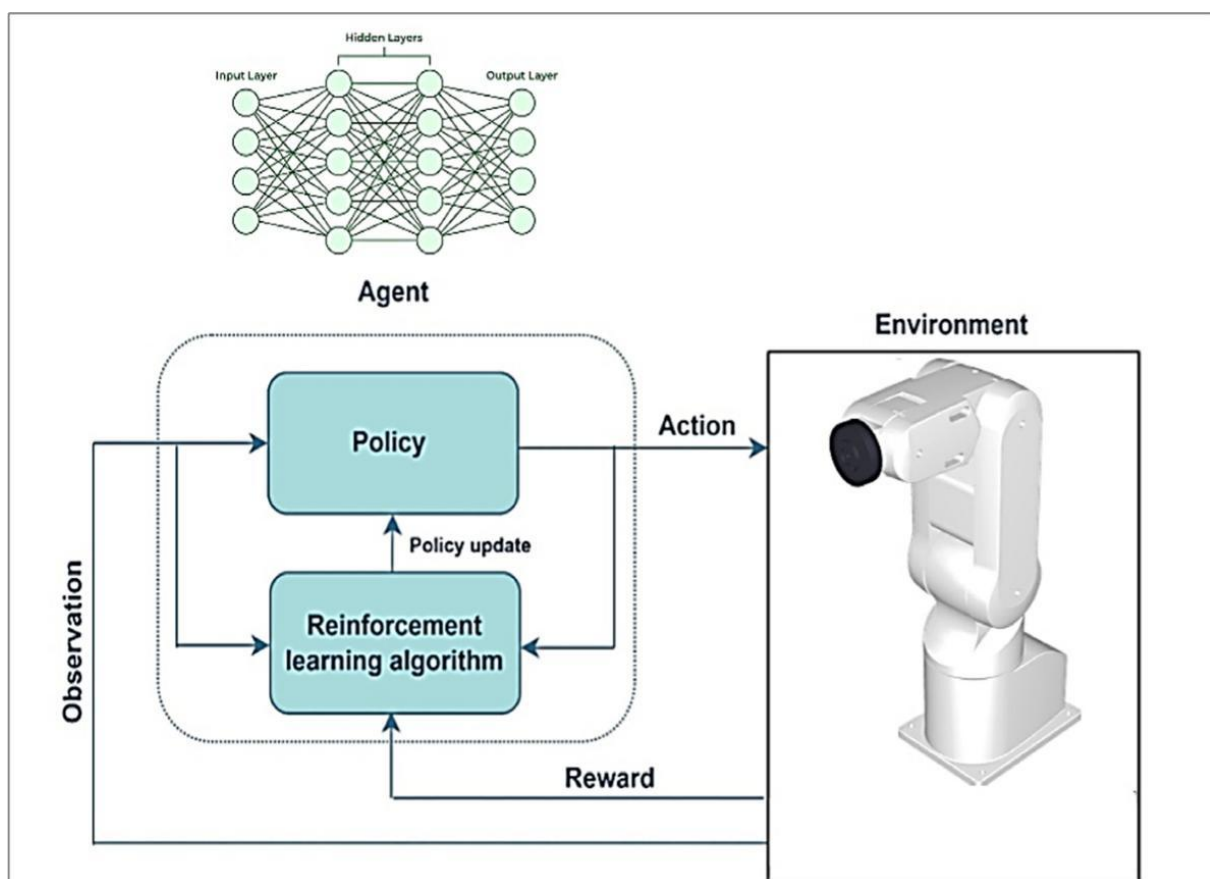
Step penalty = -1

The agent may find a shorter route.

However, if an unsafe shortcut is also highly rewarded, the agent may learn an undesirable policy.

Therefore, reward shaping should encourage **safe and goal-oriented behavior** without creating misleading incentives.

Diagram



6. Challenges of RL in Real-World Robotics

Applying RL to physical robots is more difficult than applying it in simulations.

1. Safety

A robot may damage itself, equipment, or people while exploring.

Example:

Robot explores → Takes dangerous action

↓

Collision/Damage

Therefore, safety constraints are required.

2. Sample Efficiency

RL may require thousands or millions of interactions to learn.

In simulation, this is inexpensive.

In a real robot:

1 experiment = Time + Energy + Wear + Risk

Therefore, collecting training data can be expensive.

3. Environment Variability

Real-world environments are unpredictable.

Examples:

- ◆ Lighting changes.
- ◆ Objects move
- ◆ Sensors contain noise.
- ◆ Floor conditions change.
- ◆ Obstacles appear unexpectedly

A policy learned in one environment may perform poorly in another.

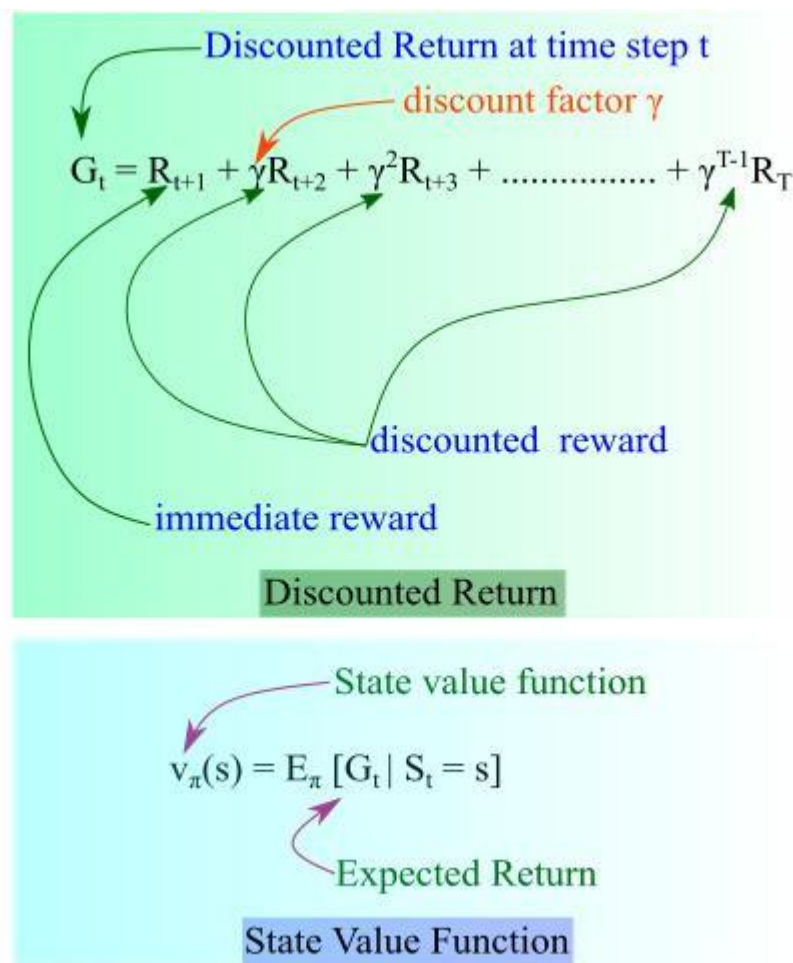
Overall Impact

These challenges can cause:

- ◆ Slow training.
- ◆ Unstable learning.
- ◆ Unsafe actions.
- ◆ Poor generalization.

Therefore, techniques such as **simulation training**, **transfer learning**, **safety constraints**, and **domain randomization** are commonly used.

Diagram



7. Influence of Discount Factor γ

The **discount factor** γ determines how much importance an RL agent gives to future rewards.

$$[0 \leq \gamma \leq 1]$$

The return is:

$$[G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots]$$

Low γ

When γ is close to 0:

Immediate reward $\rightarrow$ High importance

Future reward $\rightarrow$ Low importance

The agent becomes more **short-term oriented**.

Example:

$$[\gamma = 0.1]$$

The agent mainly considers immediate benefits.

High γ

When γ is close to 1:

Immediate reward $\rightarrow$ Important

Future reward $\rightarrow$ Very important

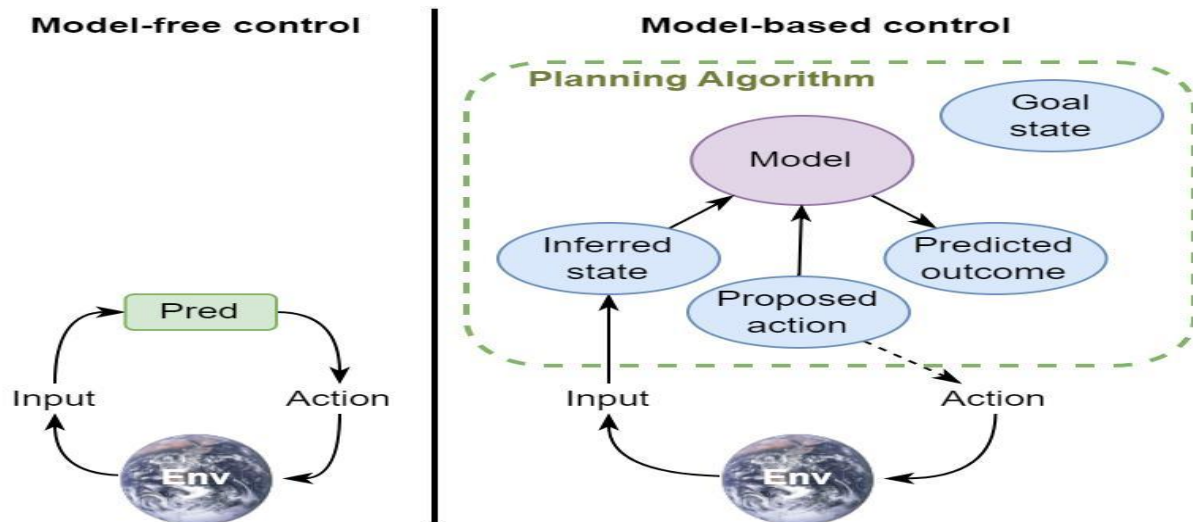
The agent becomes more **long-term oriented**.

Example:

[
\gamma=0.9
]

The agent may accept a small immediate loss for a larger future reward.

Diagram



Role in Episodic Tasks

In episodic tasks, such as a game or maze, the agent eventually reaches a terminal state.

A high γ can help the agent value rewards obtained near the end of an episode.

A low γ can make the agent focus mainly on immediate rewards.

Therefore, choosing γ appropriately is important for achieving the desired balance between short-term and long-term decision-making.

8. Model-Free RL vs Model-Based RL

Model-Free RL

Model-free algorithms learn directly from interaction with the environment without explicitly learning the environment's transition model.

Examples:

Q-learning

SARSA

DQN

State $\rightarrow$ Action $\rightarrow$ Reward
 $\downarrow$
Learn Q/Policy

Model-Based RL

Model-based RL learns or uses a model of the environment.

The model estimates:

[
 $P(s'|s,a)$
]

and

[
 $R(s,a)$
]

The agent can then predict future states before taking actions.

State $\rightarrow$ Model $\rightarrow$ Predict future
 $\downarrow$
Choose action

Comparison

Model-Free

Does not require environment model

Simple concept

Often requires many samples

Good when model is difficult to learn

Can adapt directly from experience

Model-Based

Uses/learns environment model

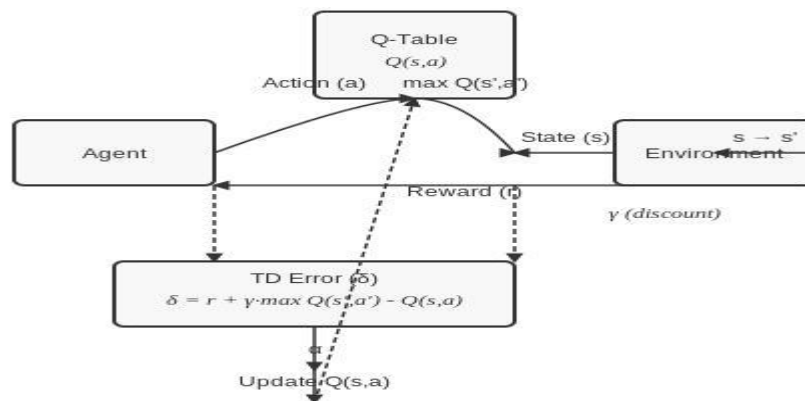
More complex

More sample efficient

Good when predictions are accurate

Model errors can affect decisions

Diagram



Dynamic Environments

In highly dynamic environments, model-free methods can be useful because they learn directly from new experiences.

Model-based methods can be more efficient when the environment can be modeled accurately.

Therefore, neither approach is universally superior; the choice depends on **environment complexity, predictability, and available training data.**

9. Q-Learning Update Example

Q-learning is an off-policy, model-free RL algorithm that learns the optimal action-value function.

The Q-learning update rule is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

α = learning rate

γ = discount factor

r = immediate reward

s' = next state

Example

Suppose:

Current state = S1

Action = Right

Reward = 10

Next state = S2

Assume:

$$Q(S1, Right) = 2$$

$$[\backslash \max Q(S_2, a') = 8]$$

and:

$$[\backslash \alpha = 0.5, \backslash \text{quad} \backslash \gamma = 0.9]$$

Then:

$$[\begin{aligned} &Q(S_1, \text{Right}) \\ &= 2 + 0.5[10 + (0.9)(8) - 2] \end{aligned}]$$

$$[\begin{aligned} &= 2 + 0.5[10 + 7.2 - 2] \end{aligned}]$$

$$[\begin{aligned} &= 2 + 0.5(15.2) \end{aligned}]$$

$$[\begin{aligned} &= 9.6 \end{aligned}]$$

So the Q-value changes from:

$$[\begin{aligned} &2 \rightarrow 9.6 \end{aligned}]$$

Convergence

During training:

Initial Q-values



Exploration



Q-value updates



Better action estimates



Repeated updates

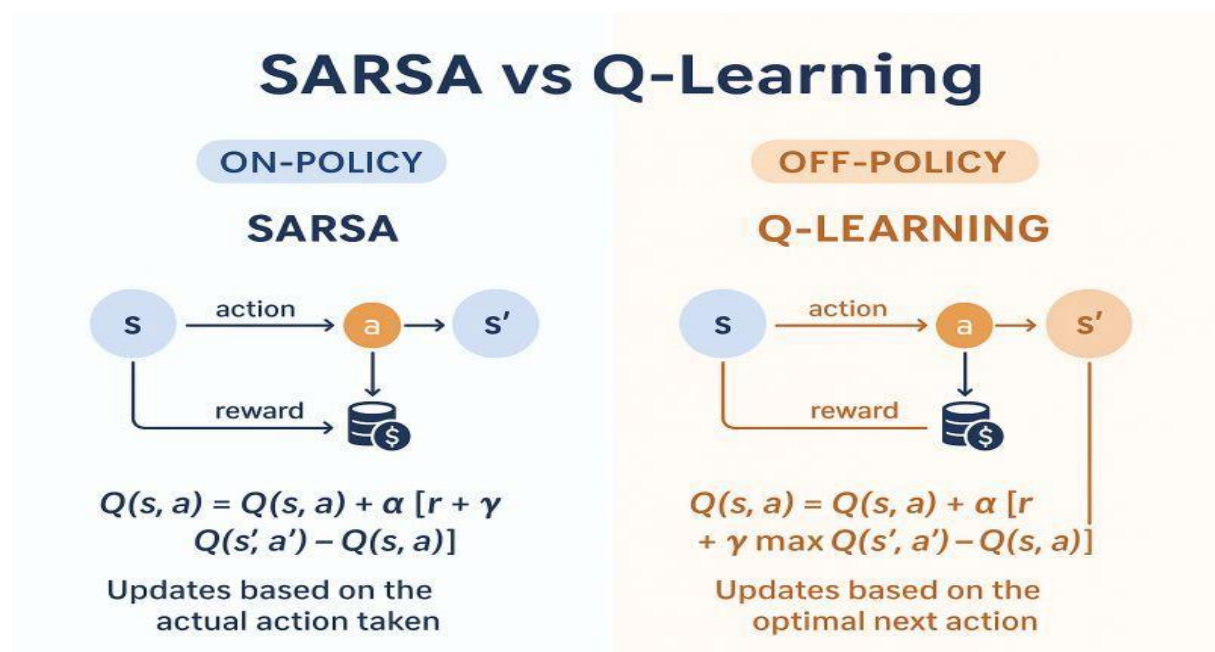


Q-values stabilize



Optimal policy

Diagram



With sufficient exploration and appropriate learning conditions, Q-learning can converge toward the optimal action-value function.

10. On-Policy vs Off-Policy Learning

The major difference is **which policy is used to generate experience and which policy is being learned.**

On-Policy Learning

An on-policy algorithm learns about the **same policy that it follows**.

Example:

SARSA

$$\begin{aligned} &[\\ &Q(s,a) \leftarrow Q(s,a) + \\ &\alpha [r + \gamma Q(s',a') - Q(s,a)] \\ &] \end{aligned}$$

Here, the next action (a') is selected according to the current policy.

Advantages

Considers the actual behavior of the agent.

Useful when exploration must be considered during learning.

Can learn safer behavior in some environments.

Limitation

Learning can be slower.

Exploration affects the learned policy.

May require more experience.

Off-Policy Learning

Off-policy algorithms learn one policy while following another policy.

Example:

Q-learning

$$[\\begin{aligned} &Q(s,a) \\leftarrow &Q(s,a)+ \\alpha [r+\\gamma \\max_{a'} Q(s',a')-Q(s,a)] \\end{aligned}]$$

The agent can explore using one behavior policy while learning the optimal policy.

Advantages

- ◆ Separates exploration from the target policy.
- ◆ Can reuse data from previous experiences.
- ◆ Often highly effective for learning optimal policies.

Limitations

- ◆ Learning can become unstable in some settings.
- ◆ Distribution mismatch between behavior and target policy can cause problems.
- ◆ Requires careful handling of exploration and training data.

Comparison

Feature	On-Policy	Off-Policy
Learns	Current behavior policy	Target policy
Example	SARSA	Q-learning
Exploration	Part of learned behavior	Can be separate
Flexibility	Lower	Higher
Data reuse	More limited	Better
Practical use	Safety-sensitive behavior	Large-scale learning

Diagram

Conclusion

On-policy methods are useful when the agent should learn from the behavior it actually follows, while **off-policy methods** are useful when learning an optimal policy from different or previously collected experiences.

Quick Revision Table

Q	Main Concept	Key Formula/Term
1	RL Framework	State, Action, Reward
2	MDP	(S, A, P, R, γ)
3	Policy & Value	Bellman Equation
4	Exploration	ϵ -greedy, Softmax
5	Reward Shaping	Additional rewards
6	Robotics	Safety, samples, variability
7	Discount Factor	$(0 \leq \gamma \leq 1)$
8	RL Types	Model-free vs Model-based
9	Q-learning	Q-update equation
10	Learning Policies	On-policy vs Off-policy